



< Previous



Next >

Complexity and Structure

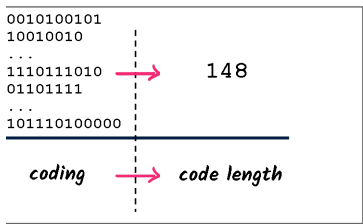
Bookmark this page

Problem: Can we define the complexity of new objects, based on their structure?

as.

```
, size(4),
), size(4),
```

an then be converted
eans) into binary
is binary form gives
plexity of the object.



f estimating
on consists of
... ..

For now, we are able to estimate the complexity of integers and of objects that are stored in memory (through the complexity of their address). What about new objects?

Consider, for instance, the case of passwords. When looking for a good password, we try to outsmart machines. This situation offers a nice illustration of the importance of complexity.

Question: Complex passwords are impossible to remember. Can we define what a good password should be?

Answer: A good password should be easy to remember, but hard to guess. In AIT’s terms, it should be *complex* to anyone but you.

Password complexity

6/6 points (ungraded)

How would you *rank* the complexity of the following passwords (from 1=simplest to 6=most complex)?

efghijkl

✓

aabbccdd

✓

abcdefgh

✓

amoijsdf



aaaaaaaa



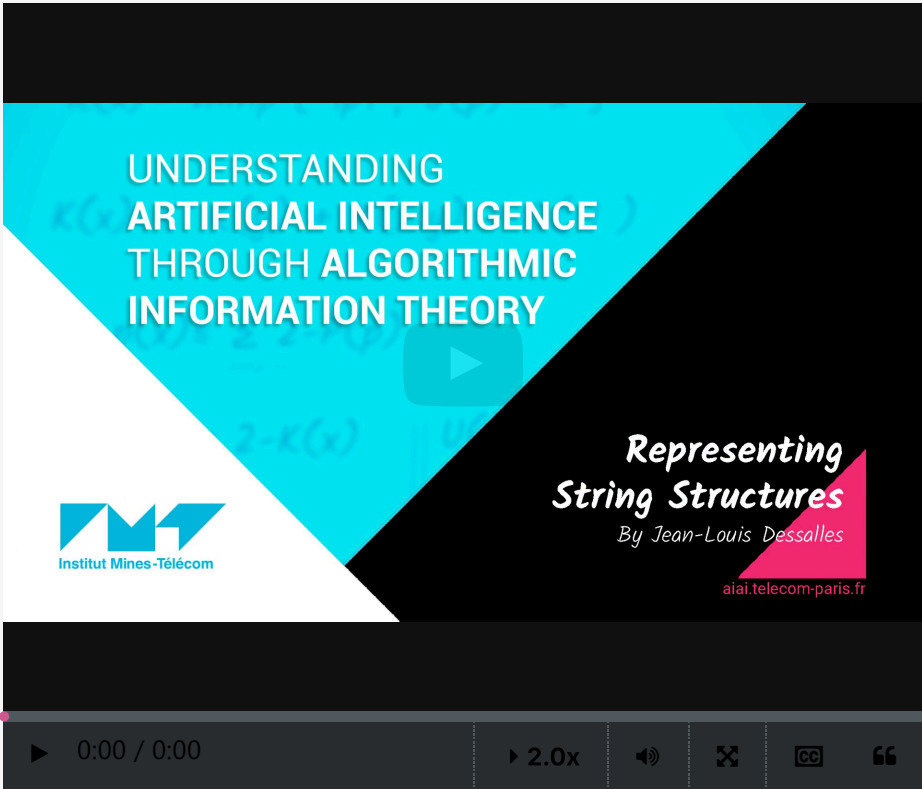
abcdabcd



You have used 1 of 1 attempt

Passwords are character strings. Some strings are obviously simpler than others. How can we quantify this? A first step consists of finding a compact formal representation of the string, such as `repetition(a, 8)`, and then translate this formal representation into bits using some appropriate code, and finally counts the bits.

String Structures



[Start of transcript. Skip to the end.](#)

Measuring the complexity of objects supposes that we can represent them as short binary strings But the theory does not say how to proceed. What is the shortest binary string that

▶ String Structures in a nutshell

Simple Structure 1

0/1 point (ungraded)

Using the coding scheme presented in the video, determine which string is represented by: `mapping(increment(1,a,4), increment(1,X,2))`.

- ☐ cdef
- ☒ aabbccdd
- ☐ aaaaaaaaa
- ☐ abbccdde
- ☐ abcdabcd

○ abcccdde

○ ee

Answer

Incorrect:
mapping(increment(1,a,4), increment(1,X,2)) is the code for abbccdde.
increment(1,a,4) means abcd and increment(1,X,2), when mapped on abcd, produces ab bc cd de.

Submit

You have used 1 of 1 attempt

 Answers are displayed within the problem

Simple Structure 2

1/1 point (ungraded)

Provide an expression that represents `abcdefgh` .

increment(1,a,8)

✓ Answer: `increment.1,\s*'?'a'?,\s*8.`

`abcdefgh --> increment(1,a,8)`

Submit

You have used 1 of 1 attempt

 Answers are displayed within the problem

<i>letter</i>	<i>position</i>	<i>code</i>
a	1	0
b	2	1
c	3	00
d	4	01
e	5	10
f	6	11
g	7	000
h	8	001

Suppose we want to measure the complexity of a password such as `abcdefgh`. We can code for the letters one by one. The codes that appear in the video assume standard binary code. We can get a slightly more compact code using our compact integer coding scheme (see table).

If we add up all the bits used in codes, we get a total of 16 bits for `abcdefgh` (not counting space delimiters). But we can do better than that, if we consider that `abcdefgh` can be represented as `increment(1, a, 8)`. To do so, suppose that we limit ourselves to the following list of available operators, with the corresponding codes.

operator:	repetition	increment	mapping
code:	0	1	00

Suppose now that we consider three kinds of entities, coded accordingly:

entity:	operator	integer	letter
code:	0	1	00

Using this coding scheme, an object like letter `e` will be coded as `00 10`, where the first two bits designate `letters` and the last two bits designate the rank of `e` among letters.

Using the code 1

1/1 point (ungraded)

What would be the correct code to designate number 6 in our small language?

- ☐ 000
- ☐ 110
- ☒ 1 000
- ☐ 1 110
- ☐ 00 000
- ☐ 11 110

✓

Answer
Correct:
The number 6 is represented by the code: 1 000, where 1 selects integers, and 000 is the compact code of 6 (binary code of 6+2 without heading 1).

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

Using the code 2

0/1 point (ungraded)

What would be the correct code to designate operator `mapping` in our small language?

- ☒ 0 00
✓
- ☐ 0 0 00
- ☐ 000
- ☐ 00

Answer
Incorrect:
The operator increment is represented by the code: 0 00, where 0 selects operators, and 00 designates mapping among operators.

You have used 1 of 1 attempt

 Answers are displayed within the problem

Using the code 3

1/1 point (ungraded)

What would be the correct code to designate letter `b` in our small language?

☐ 10

☐ 00 11

☒ 00 1

☐ 00



Answer
Correct:
The letter `b` is represented by the code: 00 1, where 00 selects letters, and 1 designates `b` among letters.

You have used 1 of 1 attempt

 Answers are displayed within the problem

Since we are able to represent `abcdefgh` as `increment (1, a, 8)`, we can now translate it into a binary representation. We still use our compact integer coding system. Note that when the type of the object is known, we can avoid using a complete code and thus get an overall shorter representation.

<i>element</i>	<i>code</i>	<i>comment</i>
_____	_____	_____
increment	0 1	1 = designation of <code>increment</code> as operator
1	1	an integer is expected
a	00 0	complete code is expected (here: letter + a)
8	010	an integer is expected
_____	_____	_____

We end up with the following code: 0 1 1 00 0 010 to represent `abcdefgh`. It is 9 bits long (ignoring space delimiters), which is more compact than the 16 bits of the letter-by-letter code (note: the code is decodable only if we keep the spaces between code words). Our coding scheme is sufficient to provide an estimate of the description complexity of alphabetic strings.

Description length of `abcdabcd`

1/1 point (ungraded)

Using the same coding method, provide a binary representation of `abcdabcd` ?

0 0 0 1 1 0 0 0 1 0 0 0

✔ Answer: 0 0 0 1 1 0 0 0 1 0 0 0

Answer

Correct:

As we saw, `abcdabcd` can be represented by: `repetition(increment(1,a,4),2)` .

This representation is then coded as:

- repetition --> `0 0`
- increment --> `0 1` (complete code expected)
- 1 --> `1` (integer expected)
- a --> `00 0` (complete code expected)
- 4 --> `10` (integer expected)
- 2 --> `00` (integer expected)

The resulting code `0 0 0 1 1 0 0 0 1 0 0 0` is **12 bit long**.

Submit

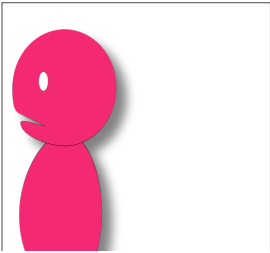
You have used 1 of 1 attempt

i Answers are displayed within the problem

Note that the resulting description length of `abcdabcd` is larger than the description length of `abcdefgh`. This might explain why `abcdabcd` is perceived as more complex than `abcdefgh`.



So complex integers must be far away from simple ones?



< Previous

Next >

Legal

Connect

